

# Flight Path — Phase 0 Summary (Plain English)

---

**For:** Jonathan (owner, non-engineer)

**What this is:** A complete, jargon-light tour of the app as it exists today, how to run it on a Mac, and exactly where the three new features (user management, profiles, and AI chat) will plug in.

**Important:** Phase 0 made **no code changes**. This is purely a read-and-understand pass so we all start from the same map.

---

## 1. The one-paragraph version

---

Flight Path is a **private, login-protected website that republishes your Notion content** (the “Flight Path Program” wiki) in a polished dark style — think of it like Super.so, but it’s our own app sitting behind a company login. Notion stays the place where you **write and edit**; this app just **displays** what’s in Notion, fast and beautiful, only to people you’ve allowed in. There’s also a matching **iOS app** and a background **crawler** that copies Notion content into a local database the website reads from.

---

## 2. What’s actually in the box (the three programs)

---

The project is a “monorepo” — one folder holding three separate programs that work together:

| Folder                    | What it is                                       | Plain-English job                                                                                                  |
|---------------------------|--------------------------------------------------|--------------------------------------------------------------------------------------------------------------------|
| <code>web/</code>         | The website <b>and</b> the behind-the-scenes API | What people log into and read. Also serves data to the iOS app. This is where almost all the new work will happen. |
| <code>node-worker/</code> | The Notion crawler                               | A background helper that reads your Notion wiki and copies every page into the local database. Run on demand.      |
| <code>ios/</code>         | The iPhone app (SwiftUI)                         | A native app that shows the same content. Not the focus of the next features.                                      |
| <code>db/</code>          | The database blueprint                           | A single SQL file that creates all the tables and seeds you in as the first admin.                                 |

Supporting files at the root: `CLAUDE.md` (the master spec for AI agents), `README.md`, `SETUP_GUIDE.md`, `AUTH_SETUP.md`, and `docker-compose.yml` (starts the database).

⚠ **Heads-up about the docs vs. the code.** `CLAUDE.md`, `README.md`, and `SETUP_GUIDE.md` were written early and still mention **Supabase** and **magic-link email login**. The code has since **moved on**: there is **no Supabase anymore**. The app now uses a **local PostgreSQL database** (in Docker) and **“Sign in with Google”**. The accurate, up-to-date doc is `web/AUTH_SETUP.md`. When the older docs and the code disagree, **the code wins**. (Cleaning up those stale docs is a good small task for later.)

### 3. The big picture (how a page gets to a reader)

```

You edit content in Notion → node-worker crawls it → Local PostgreSQL DB
(the "Flight Path Program" wiki)   (copies pages in)   (stores the pages)
                                     |
                                     the website reads from the DB
                                     |
Reader signs in with Google → passes the "gate" → sees the content (dark UI)

```

Two things to notice:

1. **The website reads from the database, not live from Notion.** That's what makes it fast. To show new Notion edits, the crawler has to run again (a “sync”).
2. **Everything is behind the login gate.** Nothing is public.

## 4. The current tech stack (confirmed)

- **Website + API:** Next.js 16 (App Router) + React 19 + TypeScript, styled with Tailwind CSS v4.
- **Login: NextAuth v5** (also called Auth.js beta) using **Google sign-in**, plus a **custom gate** that decides who's actually allowed in. Sessions are **JWT cookies** (no database lookup on every page — the user's id + role are baked into the cookie at sign-in).
- **Database: Local PostgreSQL** running in Docker (port **5433**, container name `flightpath-postgres`). The blueprint is `db/init/01-schema.sql`.
- **Notion:** the official `@notionhq/client`, used **only** by the `node-worker` crawler.
- **iOS:** Swift + SwiftUI (separate, not part of the next features).

## 5. How each important piece works

### 5a. Login + the “gate” (the heart of access control)

Two files do the work:

- `web/src/auth.ts` — sets up NextAuth with Google as the only sign-in method. When someone signs in, Google **proves who they are** (verifies they own the email). It does **not** decide whether they're allowed.
- `web/src/lib/auth/gate.ts` — this is **the gate**: it decides who is allowed in. The rule is:
  1. The email's **domain** must be in the `allowed_domains` table (**e.g. `sunritesolarllc.com`**), **AND**
  2. If “invite required” is turned on, the **exact email** must be in the `invites` table.

If either check fails, the sign-in is **denied** and the person is bounced back to the landing page with a friendly “Access denied” message.

Key design point the spec insists on: **allowed domains and invites live in the database, never hard-coded**. That way you can change who's allowed without touching code.

On first successful sign-in, the app creates a row for the user in `app_users` and remembers their **role** (`member` or `admin`). Your email (`jrizzo@sunritesolarllc.com`) is **pre-seeded as an admin** by the database blueprint, so you're ready to go on day one.

### 5b. Notion content (the crawler / “sync”)

`node-worker/` reads your Notion wiki and copies it into the database:

- It starts at one root page/database (the **“Flight Path Program”** wiki), set via the `NOTION_ROOT_PAGE_ID` environment variable.
- It **crawls recursively** — the root, its pages, their sub-pages, to a configurable depth.
- For each page it grabs the title, icon, cover, and the full body, then **“normalizes”** the messy Notion blocks into a clean, predictable shape (`normalize.ts`) that the website knows how to render (headings, paragraphs, lists, toggles, callouts, quotes, code, images, bookmarks, files, page links, dividers).
- It writes everything into the `notion_pages` table. The website reads from there.

### Two known gaps already flagged in the code (not bugs you caused):

- The **“Hidden Pages”** rule isn’t fully built yet. The spec says: a page tucked inside a Notion toggle literally labeled “Hidden Pages” should be hidden from browsing but still reachable by direct link. Today the crawler uses a rough placeholder (it guesses based on title/tag keywords like “hidden”/“premium”). This is marked `TODO` in `notion.service.ts`.
- **Deleted Notion pages** aren’t cleaned out of the database yet (also a `TODO`).

## 5c. Navigation

There is **no fancy navigation tree yet**. Right now the website shows a **flat library**:

`/pages` lists every non-hidden page as a card, and clicking one opens `/pages/[slug]`.

The richer nested side-navigation described in the spec ( `GET /api/site` returning a tree) is **not built yet** — it’s a future phase. Good to know so we don’t assume it exists.

## 5d. The “Azurio” dark styling

The dark, editorial look is currently achieved with **Tailwind utility classes applied directly inside the page files** — dark slate gradient backgrounds

( `from-slate-950 via-slate-900` ), **orange** accent ( `orange-500` ), soft blurred accent glows, thin white borders, rounded cards. There is **no dedicated Azurio CSS/theme file yet** ( `web/src/app/globals.css` is still close to the Next.js default). So “Azurio” today = a consistent set of Tailwind classes repeated across pages, not a centralized design system. Worth knowing if we want a shared theme later.

---

## 6. The pages and API that exist today

---

**Pages people can visit (all require login except the landing page):**

| URL           | File                      | What it does                                                                                                                                                                          |
|---------------|---------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| /             | app/page.tsx              | Landing page + “Sign in with Google” button. Shows “Access denied” if the gate rejects you. Logged-in users skip straight to /pages .                                                 |
| /pages        | app/pages/page.tsx        | The content library — a grid of all non-hidden pages.                                                                                                                                 |
| /pages/[slug] | app/pages/[slug]/page.tsx | One full page, with all its Notion blocks rendered. Hidden pages are reachable here by direct link.                                                                                   |
| /admin        | app/admin/page.tsx        | Admin-only dashboard: page counts, last sync time, and a <b>read-only</b> view of allowed domains + invites. (The add/remove buttons are not built yet — currently you edit via SQL.) |

#### Behind-the-scenes API:

| Route       | File                                | What it does                                                                                                 |
|-------------|-------------------------------------|--------------------------------------------------------------------------------------------------------------|
| /api/auth/* | app/api/auth/[...nextauth]/route.ts | Handles the Google sign-in handshake.                                                                        |
| /api/pages  | app/api/pages/route.ts              | Returns the list of pages (or one page by ?slug= ). Requires a valid session (returns 401 if not logged in). |

**Helpers:** web/src/lib/db.ts (the database connection), web/src/lib/types/index.ts (the shared data shapes), web/src/types/next-auth.d.ts (adds id + role to the session).

## 7. The database tables (what’s stored)

Created automatically by db/init/01-schema.sql the first time the database starts:

- **app\_users** — people who can sign in. Has a role column: 'member' or 'admin'. (This table is central to the upcoming user-management and profiles features.)

- `allowed_domains` — which email domains may sign in (seeded with `sunritesolarllc.com`).
- `invites` — specific invited email addresses (you're pre-invited and accepted).
- `login_tokens` / `sessions` — supporting tables for sign-in (the JWT-cookie approach means `sessions` is largely unused today).
- `notion_pages` — the cached copy of every crawled Notion page (this is what the website displays). Has an `is_hidden` flag.
- `page_versions` — snapshots of page content for change history.
- `crawl_logs` / `sync_meta` — crawler history and the "last sync" badge on the admin page.
- `admin_settings` — global settings (e.g. the "invite required" on/off switch).

## 8. The npm commands (cheat sheet)

### Website ( `web/` ):

```
npm install      # install the website's dependencies (first time only)
npm run dev      # start the website in development mode
npm run build    # make a production build
npm start        # run the production build
npm run lint     # check code style
```

### Crawler ( `node-worker/` ):

```
npm install      # install the crawler's dependencies (first time only)
npm run dev      # run the crawler in watch mode
npm run build    # compile it
npm start        # run the compiled crawler
```

### Database (from the repo root):

```
docker compose up -d  # start the local PostgreSQL database
docker compose ps      # check it's running
docker compose down    # stop it (keeps your data)
```

## 9. Environment variables (the settings you must fill in)

These are secret settings the app reads at startup. They live in files that are **never committed** (gitignored). There are three places:

**A) Repo root** `.env` (for the database, read by `docker-compose.yml` ):

```
POSTGRES_PORT=5433
POSTGRES_DB=flightpath
POSTGRES_USER=flightpath
POSTGRES_PASSWORD=<a long random password>
DATABASE_URL=postgres://flightpath:<that same password>@localhost:5433/flightpath
```

**B) web/.env.local** (for the website):

```
DATABASE_URL=postgres://flightpath:<password>@localhost:5433/flightpath
AUTH_SECRET=<a long random string> # signs the login cookies
AUTH_GOOGLE_ID=<from Google Cloud>.apps.googleusercontent.com
AUTH_GOOGLE_SECRET=<from Google Cloud>
```

**C) node-worker/.env** (for the crawler):

```
NOTION_API_KEY=<your Notion integration secret>
NOTION_ROOT_PAGE_ID=<the Flight Path Program wiki id>
DATABASE_URL=postgres://flightpath:<password>@localhost:5433/flightpath
```

 **Note:** the repo does **not** include a `web/.env.example` template (even though `CLAUDE.md` mentions one). Creating one is a nice small cleanup task. The crawler does have a `node-worker/.env.example`.

## 10. What I verified actually works (Phase 0 smoke test)

On a clean machine I confirmed end-to-end:

-  The repo **clones** cleanly.
-  `web/` dependencies **install** (391 packages, no blocking errors).
-  `node-worker/` dependencies **install** (0 vulnerabilities).
-  **Both** projects **pass TypeScript type-checking** with zero errors.
-  The website **dev server starts** (Next.js 16, ready in ~0.25s) and the **landing page loads** with the “Sign in with Google” button.
-  The **login gate works**: visiting `/pages` without logging in correctly **redirects** to the landing page, and `/api/pages` correctly returns **401 Unauthorized**.
-  The **database schema loads** and **seeds you** ( `jrizzo@sunritesolarllc.com` ) as the admin, with `sunritesolarllc.com` as the allowed domain.

**The one thing that needs your input to fully sign in: real Google OAuth credentials** ( `AUTH_GOOGLE_ID` and `AUTH_GOOGLE_SECRET` ). Those come from a 5-minute setup in the Google Cloud Console — the exact click-by-click steps are in `web/AUTH_SETUP.md` . Without them, everything runs but the actual Google sign-in button can’t complete. (To see real content you’d also need a Notion API key + the wiki page id for the crawler.)

Note about this environment: I ran the smoke test on the Abacus AI Agent computer (a cloud VM I use to run the app), **not** your Mac. Because Docker wasn’t available on that VM, I started a local PostgreSQL on port 5433 to stand in for the Docker database — the app behaved identically. On your Mac you’ll use `docker compose up -d` as documented.

## 11. How to run it on your Mac (plain steps)

1. **Install the basics:** Docker Desktop, Node.js v20+, and Git.

2. **Get the code:** `git clone https://github.com/jtrizz-ai/flight-path-by-sunrite.git`
3. **Create the root `.env`** (copy `.env.example` to `.env`, set a real password).
4. **Start the database:** from the repo root, `docker compose up -d`.
5. **Set up Google sign-in:** follow `web/AUTH_SETUP.md` to get your `AUTH_GOOGLER_ID` / `AUTH_GOOGLER_SECRET`, and put them (plus `DATABASE_URL` and a random `AUTH_SECRET`) into `web/.env.local`.
6. **Run the website:** `cd web`, then `npm install`, then `npm run dev`. Open the URL it prints (the auth doc uses port **3100** because port 3000 may be taken by Grafana on your Mac).
7. **(To load content)** set up `node-worker/.env` with your Notion key + wiki id, then `cd node-worker && npm install && npm run dev` to crawl Notion into the database.
8. Sign in with `jrizzo@sunritesolarllc.com` — you're already an invited admin.

## 12. Where the THREE new features will plug in

Here's the map for the work ahead — what each feature touches and what already exists to build on.

### Feature 1 — User management (admin can add/remove people & roles)

- **Already there to build on:** the `app_users`, `allowed_domains`, and `invites` tables; the gate logic in `gate.ts`; the admin page at `/admin` (which today only displays these lists, read-only); and the admin-only access check (`session.user.role === "admin"`).
- **Where it plugs in:**
  - New API routes under `web/src/app/api/admin/` — e.g. `domains/route.ts` and `invites/route.ts` — to **add/remove** domains and invites (the spec even names these).
  - Possibly an `app_users` route to **change a user's role** (promote to admin / demote).
  - Turn the read-only lists on `/admin/page.tsx` into **interactive add/remove buttons**.
- **Why it's a clean fit:** the database and the gate already read from these tables, so we only need to add the "write" side (create/delete) plus buttons in the admin UI.

### Feature 2 — User profiles

- **Already there to build on:** `app_users` already stores `email`, `full_name`, `avatar_url`, `role`, `created_at`, and `last_active_at`. The session already carries the user's `id` and `role`.
- **Where it plugs in:**
  - A new page, e.g. `web/src/app/profile/page.tsx`, where a logged-in user views/edits their own profile.
  - A new API route, e.g. `web/src/app/api/profile/route.ts`, to read and update the user's row in `app_users`.
  - Likely a few **new columns** on `app_users` (e.g. job title, bio, preferences) added via a new SQL migration file in `db/init/`.
- **Why it's a clean fit:** the user record and session plumbing already exist; profiles mostly extend the `app_users` table and add a couple of screens.



## Feature 3 — AI chat (read-only assistant over your content)

- **Already there to build on:** all your content is already crawled, normalized, and stored in `notion_pages` (with a clean `blocks` body and an `is_hidden` flag). The spec has a detailed, safety-first design for this (CLAUDE.md sections 9 & 14).
- **Where it plugs in:**
  - New API routes: `web/src/app/api/chat/route.ts` (ask a question), `chat/health/route.ts` (is the AI reachable?), and `chat/reindex/route.ts` (rebuild the search index).
  - New helper code under `web/src/lib/chat/` — retrieve the most relevant chunks of content, build a strict prompt, and enforce guardrails.
  - A chat UI (a panel or a `/chat` page in `web/`, plus the existing `ChatView` in iOS).
- **Non-negotiable guardrails from the spec:** the assistant is **read-only** (it can never change Notion), it must **only** answer from your content (and say so when it doesn't know), it must **never** see **hidden** pages, and the local AI model is reached over a private network — never exposed publicly.
- **Why it's a clean fit:** the hard part (getting clean content into a database) is done; chat is mostly retrieval + prompting + a UI on top of `notion_pages`.

## 13. Honest list of gaps & cleanups (so nothing surprises us later)

1. **Stale docs:** `CLAUDE.md`, `README.md`, `SETUP_GUIDE.md` still describe Supabase + magic-link. Reality is local Postgres + Google sign-in ( `web/AUTH_SETUP.md` is correct).
2. **“Hidden Pages” rule** is only a placeholder heuristic, not the real toggle-based rule.
3. **No GET /api/site nav tree** yet — the library is flat, not nested.
4. **Admin portal is read-only** — no add/remove buttons yet (Feature 1 fixes this).
5. **Deleted Notion pages** linger in the database (cleanup `TODO` ).
6. **No `web/.env.example`** template committed.
7. **No centralized “Azurio” theme file** — styling is repeated Tailwind classes.
8. **iOS app** still references older “modules” naming in places; it's not part of the three new features, so we can leave it for now.

None of these block the next phase. They're just the honest state of things so we plan with eyes open.

Prepared in Phase 0. No code was changed. Next step: decide which of the three features to build first — user management is the most natural starting point because the tables and admin page already exist and only need the “add/remove” actions wired up.